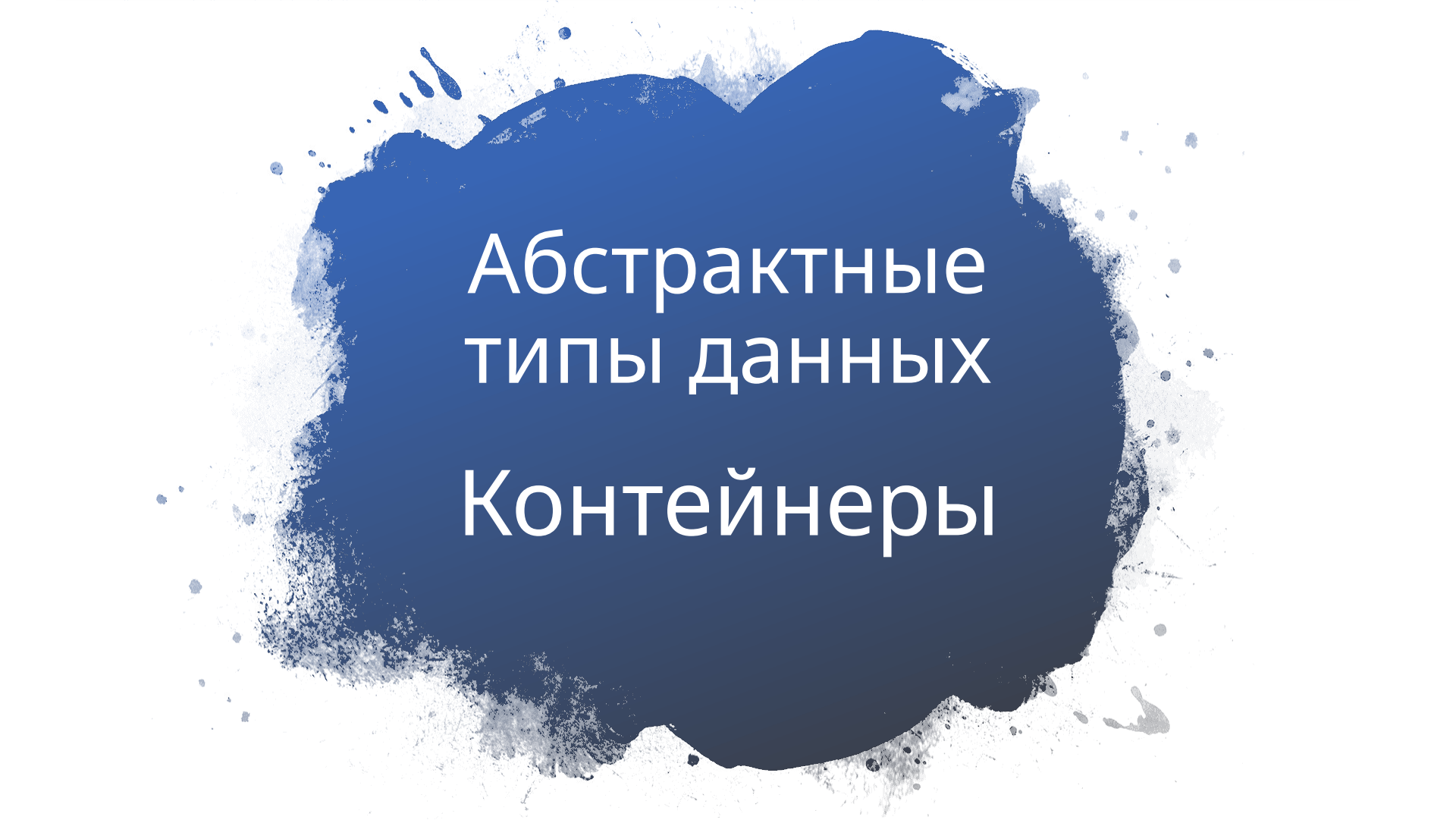


The background of the slide is a white surface with a large, irregular, dark blue ink splash or blotch in the center. The splash has a textured, painterly appearance with some lighter blue and white speckles around its edges. The text is centered within this blue area.

Абстрактные типы данных Контейнеры

Абстракция

- 1. Абстракция** - подразумевает собой процесс изменения уровня детализации программы. Когда мы абстрагируемся от проблемы, мы предполагаем игнорирование ряда подробностей с тем, чтобы свести задачу к более простой.
2. Примером абстракции может служить использование функций и классов, реализованных в сторонней библиотеке. Мы не задумываемся о том, как реализована та или иная функция или метод, а просто используем их для решения собственных задач.

Определение абстрактного типа данных

1. Абстрактный тип данных (АТД) - совокупность данных и выполняемых над ними операций. Операции описывают данные для остальной части программы и позволяют их изменять.

1. АТД включает в себя абстракцию как через параметризацию, так и через спецификацию.

Абстракция через параметризацию

Абстракция через параметризацию (АП) позволяет, используя параметры, представить набор различных операций одной программой, которая является абстракцией всех этих наборов. Аппарат формальных и фактических параметров процедур в языках высокого уровня осуществляет АП.

Абстракция через параметризацию

```
int Array[100];  
//инициализация массива  
int max_index = 0;  
int max_value = Array[0];  
for (int i = 1; i < 100; i++)  
{  
    if (a[i] > max_value)  
    {  
        max_value = a[i];  
        max_index = i;  
    }  
}
```

```
int Array[100];  
//инициализация массива  
int max_value = GetMaxValue(a);  
int max_index = GetMaxIndex(a);
```

Абстракция через спецификацию

Абстракция через спецификацию (АС) позволяет абстрагироваться от процесса вычислений, описанных в теле процедуры, до уровня знания того, что данная процедура должна в итоге реализовать. Это достигается путем задания для каждой процедуры спецификации, описывающей эффект ее работы. При этом смысл обращения к процедуре становится ясным через анализ ее спецификации, а не тела процедуры.

Требования к АД

1. АД должен уметь работать только со своими данными.
2. АД не должны зависеть от внешнего состояния приложения.
3. Для того, чтобы получить данные используйте методы для доступа к данным или классы реализующие такую возможность.

Различие между АТД и структурами данных

Различие между абстрактными типами данных и структурами данных, которые реализуют абстрактные типы, можно пояснить на следующем примере. Абстрактный тип данных список может быть реализован при помощи массива или линейного списка с использованием различных методов динамического выделения памяти. Однако каждая реализация определяет один и тот же набор функций, который должен работать одинаково (по результату, а не по скорости) для всех реализаций.

Преимущества АТД

1. Инкапсуляция деталей реализации.
2. Снижение сложности.
3. Ограничение области использования данных.
4. Высокая информативность интерфейса.

Контейнеры

1. **Контейнер** – набор однотипных элементов. Встроенные массивы в C++ - частный случай контейнера.
2. Контейнер – это объект. Имя контейнера – это имя переменной.

Свойства контейнеров.

1. Контейнер, так же как и другие объекты, обладает временем жизни. Время жизни контейнера в общем случае не зависит от времени жизни его элементов
2. Элементами контейнера могут любые объекты, в том числе, и другие контейнеры.
3. Контейнеры могут быть фиксированного и переменного размера. В контейнере фиксированного размера число элементов постоянное, оно обычно задается при создании контейнер

Виды контейнеров

1. **LIFO**(Last In First Out - последним пришел первый вышел) - вставка и удаление осуществляется на одном конце.
2. **FIFO**(First In First Out - последним пришел последним вышел) - вставка и удаление осуществляется на разных концах

Операции контейнеров

1. Операции доступа к элементам, которые обеспечивают и операцию замены значений элементов;
2. Операции добавления и удаления элементов или групп элементов;
3. Операции поиска элементов и групп элементов;
4. Операции объединения контейнеров;
5. Специальные операции, которые зависят от вида контейнера.

Итераторы

Итератор (от англ. *iterator* — перечислитель) — интерфейс, предоставляющий доступ к элементам коллекции (массива или контейнера) и навигацию по ним.

Главное предназначение итераторов заключается в предоставлении возможности пользователю обращаться к любому элементу контейнера при сокрытии внутренней структуры контейнера от пользователя. Это позволяет контейнеру хранить элементы любым способом при допустимости работы пользователя с ним как с простой последовательностью или списком.

Свойства итераторов

1. Итератор указывает на отдельный элемент коллекции объектов (предоставляет *доступ к элементу*)
2. Содержит функции для перехода к другому элементу списка (следующему или предыдущему).
3. Контейнер, который реализует поддержку итераторов, должен предоставлять первый элемент списка, а также возможность проверить, перебраны ли все элементы контейнера (является ли итератор конечным)..

Постановка задачи

1. Определить класс-контейнер.
2. Реализовать конструкторы, деструктор, операции ввода-вывода, операцию присваивания.
3. Перегрузить операции, указанные в варианте.
4. Реализовать класс-итератор. Реализовать с его помощью операции последовательного доступа.
5. Написать тестирующую программу, иллюстрирующую выполнение операций.

Условие. Вариант 15

Класс- контейнер СПИСОК с ключевыми значениями типа `int`.

Реализовать:

`List::operator[]` – доступа по индексу;

`List::operator()` – определение размера списка;

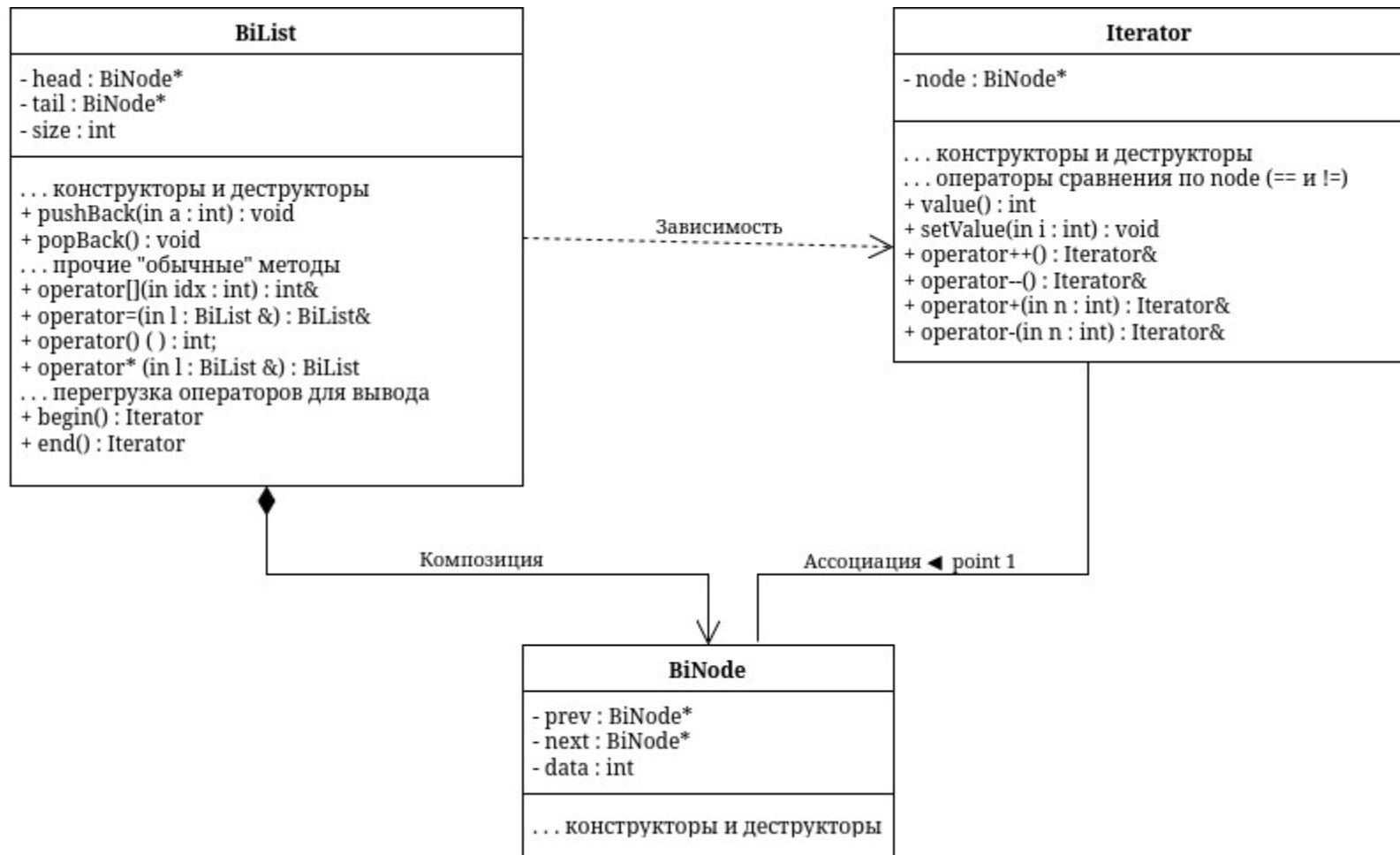
`List::operator*` – умножение элементов списков $a[i] * b[i]$;

`Iterator::operator+` - переход вправо на n элементов

BiList
- head : BiNode* - tail : BiNode* - size : int
... конструкторы и деструкторы + pushBack(in a : int) : void + popBack() : void ... прочие "обычные" методы + operator[](in idx : int) : int& + operator=(in l : BiList &) : BiList& + operator() () : int; + operator* (in l : BiList &) : BiList ... перегрузка операторов для вывода + begin() : Iterator + end() : Iterator

Iterator
- node : BiNode*
... конструкторы и деструкторы ... операторы сравнения по node (== и !=) + value() : int + setValue(in i : int) : void + operator++() : Iterator& + operator--() : Iterator& + operator+(in n : int) : Iterator& + operator-(in n : int) : Iterator&

BiNode
- prev : BiNode* - next : BiNode* - data : int
... конструкторы и деструкторы



```
class BiNode {  
    friend BiList;  
    friend Iterator;  
    BiNode *prev;  
    BiNode *next;  
    int data;  
  
    ...  
};
```

```
class BiList {
    BiNode *head;
    BiNode *tail;
    int size;

public:
    BiList();
    ...

    void pushBack(int);
    void popBack();
    ...

    int &operator[](int);
    BiList &operator=(BiList &);
    int operator()() { return size; }
    BiList operator*(BiList &);

    friend std::ostream &operator<<(std::ostream &, BiList);
    friend std::istream &operator>>(std::istream &, BiList &);

    Iterator begin() { return Iterator(head); }
    Iterator end() { return Iterator(tail->next); }
};
```

```
class Iterator {  
    BiNode *node;  
  
public:  
    ...  
    bool operator==(const Iterator &it) { return node == it.node; }  
    bool operator!=(const Iterator &it) { return node != it.node; };  
  
    Iterator &operator++();  
    Iterator &operator--();  
    Iterator &operator+(int);  
    Iterator &operator-(int);  
  
    int value() { return node->data; }  
    void setValue(int i) { node->data = i; }  
};
```

```
int main() {  
    BiList list1, list2;  
    cin >> list1 >> list2;  
    BiList list3 = list2;  
  
    for (int i = 0; i < list2(); i++)  
        list2[i] = -1;  
    cout << list2;  
  
    cout << list1 * list3;  
  
    ...  
}
```

Size? 3

Elements? 1 2 3

Size? 5

Elements? 4 5 6 7 8

-1	-1	-1	-1	-1
----	----	----	----	----

4	10	18
---	----	----

```
for (Iterator iter = list1.begin(); iter != list2.end(); ++iter)
    cout << iter.value() << ' ';          1 2 3
```

```
cout << '\n';
```

```
auto iter = list1.begin();
cout << (iter + 2).value() << ' ';          3
cout << (--iter).value() << ' ';           2
cout << (iter - 1).value() << ' ';          1
cout << '\n';
...
```



```
iter = list2.begin();
```

```
iter + 2;
```

```
iter.setValue(10);
```

```
cout << list2;
```

-1 -1 10 -1 -1